Phase trajectories of Nonlinear systems

Augmented Attractor Shaping for Nonlinear Feedback Systems

Tanush Dinesh Srivatsa
Under: Prof. Prasanta Kumar Das

Contents

1 Executive Summary & Novelty Assessment

This report outlines control architecture designed to optimize transient trajectories in highly chaotic nonlinear dynamical systems. Based on an assessment of current academic literature across reinforcement learning (RL), nonlinear dynamics, and control theory, the architecture proposed: Residual RL acting on a PID baseline, using Generalized Alignment Indices (GALI) explicitly as state inputs or offline atlases for transient trajectory optimization, represents a significant gap in the literature.

Traditionally, RL and PID are combined in robotics to handle unmodeled friction or noise. Simultaneously, chaos control leverages the Maximal Lyapunov Exponent (MLE) primarily as a reward signal (to suppress chaos). However, using a topological atlas of chaos to feed a Residual RL agent non-Markovian data specifically to optimize the transient energy of a PID-stabilized limit cycle is largely uncharted. By mathematically modeling the chaotic bounds and bridging them into real-time RL loops via sub-millisecond neural surrogates, we successfully achieve concurrent reductions in both actuator effort and tracking error.

2 Theoretical Foundation: The Physical System

2.1 The 2-Coupled Lorenz System

The benchmark physical model consists of two identical Lorenz oscillators mutually coupled via their x variables. The 6-dimensional state space is defined as $\mathbf{X} = [x_1, y_1, z_1, x_2, y_2, z_2]^T$, governed by the following nonlinear differential equations:

$$\dot{x}_1 = \sigma(y_1 - x_1) + k(x_2 - x_1) \tag{1}$$
$$\dot{y}_1 = x_1(\rho - z_1) - y_1 + u_1 \tag{2}$$
$$\dot{z}_1 = x_1 y_1 - \beta z_1 + k(z_2 - z_1) \tag{3}$$
$$\dot{x}_2 = \sigma(y_2 - x_2) + k(x_1 - x_2) \tag{4}$$
$$\dot{y}_2 = x_2(\rho - z_2) - y_2 + u_2 \tag{5}$$
$$\dot{z}_2 = x_2 y_2 - \beta z_2 + k(z_1 - z_2) \tag{6}$$

The physical constants are the standard chaotic parameters: $\sigma = 10.0, \rho = 28.0, \beta = 8/3$. The coupling strength $k = 2.5$ induces global chaotic synchronization. Control inputs u_1, u_2 are applied to the y-dimensions, with strict actuator saturation limits ($U_{max} = 250$).

2.2 Chaos Quantification: GALI vs. MLE

While the Maximum Lyapunov Exponent (MLE) identifies chaos by measuring the divergence of a single deviation vector, it fundamentally fails to capture the topological complexity of multi-scale stability limits. The Generalized Alignment Index (GALI) of order k monitors the evolution of a k-dimensional volume element. For k unit deviation vectors $\mathbf{w}_1, \ldots, \mathbf{w}_k$, the index GALI_k is defined by the norm of their wedge product:

$$\text{GALI}_k(t) = \|\mathbf{w}_1(t) \wedge \mathbf{w}_2(t) \wedge \cdots \wedge \mathbf{w}_k(t)\| \tag{7}$$

Theoretical Constraints:

- **Exponential Decay (Chaos):** If the orbit is chaotic, all vectors align with the MLE direction, causing the volume to collapse as $e^{-(\lambda_1-\lambda_k)t}$.
- **Stability:** If $k \leq N$ on a stable torus, GALI fluctuates around a non-zero constant.

In this architecture, we utilize GALI_2 (equivalent to the Smaller Alignment Index, SALI) to detect the primary transition boundary between chaotic alignment and stable vector independence.

3 Pillar 5: Dataset Generation and Manifold Extraction

3.1 The 18-Dimensional Variational System

To extract the stability manifold, exact GALI value computation across the phase space is required. This involves simulating the variational equations to evolve two deviation vectors (for $GALI_2$) alongside the 6D physical state. The combined state vector is $[\mathbf{x}, \mathbf{w}_1, \mathbf{w}_2]$, evolving via $\dot{\mathbf{w}}_k = \mathbf{J}\mathbf{w}_k$, where $\mathbf{J}$ is the 6×6 instantaneous Jacobian. This 18-dimensional integration using standard numerical methods (e.g., Runge-Kutta 45 via SciPy) is a computational bottleneck, taking $\sim$ 72ms per 10 second track of system evolution, strictly violating the latency constraints required for 500 Hz (changed later) closed-loop RL control.

3.2 Strategy C: Multi-Resolution Adaptive Sampling

To map the transition regions of chaos (regions where $0.1 < \text{GALI} < 0.4$), a 3-pass pipeline was utilized:

1. **Multi-Resolution Survey:** Latin Hypercube Sampling (LHS) with an 80/20 Core/Shell split to map dense attractors and divergence boundaries.
2. **SSM Boundary Refinement:** Targeted points at the transition edge applying local Gaussian jitter ($\sigma = 1.2$) to map the fine-grained geometry.
3. **Uncertainty-Based Surfing:** Resampling the top 10% error regions of an initial surrogate to map unvisited phase-space geometries (removed in v4).

Multiple batches of the dataset were generated, each differentiable with its sample size. Each dataset batch was tested with each iteration of the GALI surrogates. Among about ten, here are the three used sets:

- **Small**: 2,500 samples.
- **Medium** (27,500 samples): 25,000 samples + 2,500 refinement samples at transition edge.
- **Large** (325,000 samples): 250,000 samples + 75,000 refinement samples at transition edge.

4 Pillar 5: GALI Surrogate/Atlas

To resolve the integration bottleneck, successive PyTorch neural surrogates were developed that could be classified into 4 broad iterations. The goal was to map the 6D phase space directly to $\log_{10}(\text{GALI})$. The best performing models of each version are shown in the following sections.

4.1 v1 & v2: Standard MLPs and Mode Collapse

The v1 surrogate (a standard Feed-Forward MLP) achieved an 85x latency speedup (0.84ms vs 72.36ms). However, it suffered from severe mode collapse. Standard Mean Squared Error (MSE) treats all samples equally. Because chaotic regimes dominate the volumetric phase space, the network memorized deep chaos while ignoring the thin, complex transition boundaries crucial for stabilization. v2 attempted to resolve this by drastically increasing the dataset to 25,000 adaptively sampled points, yet standard MSE still allowed the chaotic bulk to overwhelm gradient updates.

4.2 v3: Physics-Informed Weighted Gradients

To force boundary resolution, v3 introduced Residual Blocks to prevent vanishing gradients across the steep exponential cliffs of Lyapunov exponents. We manually weighted the MSE loss based on physical zones:

- Deep Chaos ($\text{GALI} < 0.1$): Weight $= 1.0$
- Stable Regime ($\text{GALI} > 0.4$): Weight $= 3.75$
- Transition Zone ($0.1 < \text{GALI} < 0.4$): Weight $= 5.0$

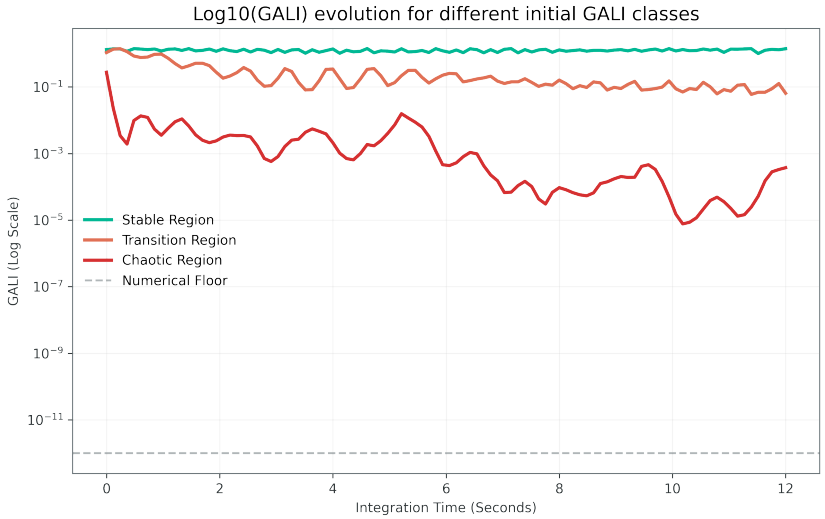

Figure 1: Temporal Evolution of $\log_{10}$(GALI) Across Dynamical Regimes. Representative trajectories illustrating stable (green), transition (orange), and chaotic (red) regimes. Stable trajectories maintain high GALI values, chaotic trajectories exhibit rapid exponential decay toward numerical floor, and transition trajectories display intermediate, fluctuating behavior. This separation justifies the definition of distinct dynamical classes used in surrogate training and adaptive sampling.

These numbers were taken based on initial GALI value track behaviour.

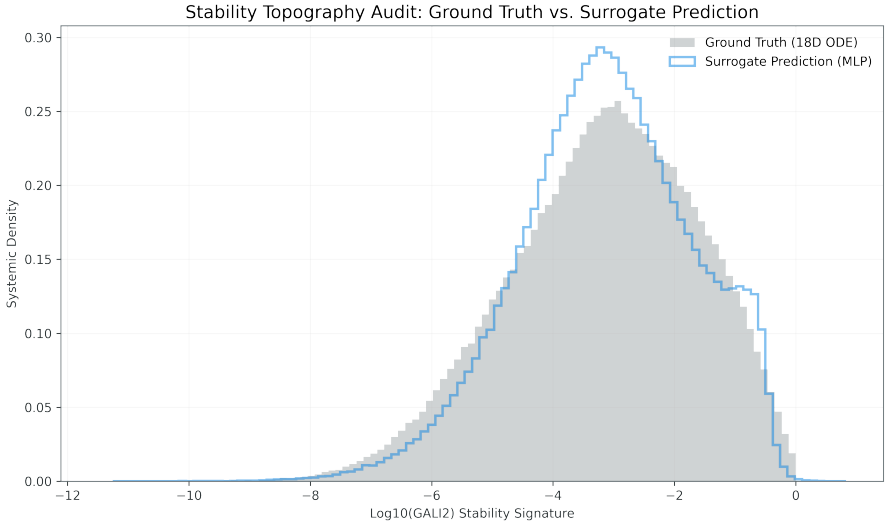

Figure 2: V3 Surrogate Prediction Audit. Histogram comparison between ground-truth $\log_{10}$(GALI) values and V3 model predictions.

Despite physics-informed weighting, the model exhibits residual bias toward high-density chaotic regions, with under-representation of the transition manifold, even after iterations of adjustment. The presence of distributional mismatch indicates incomplete resolution of thin stability boundaries. Hard-coded thresholds created artificial ridges and discontinuities in the continuous output space. The RL agent later exploited these

ridges as false local minima, leading to inaccurate residual corrections that worsened the overall augmented controller.

4.3 v4: Inverse-Density Distribution Matching (Production)

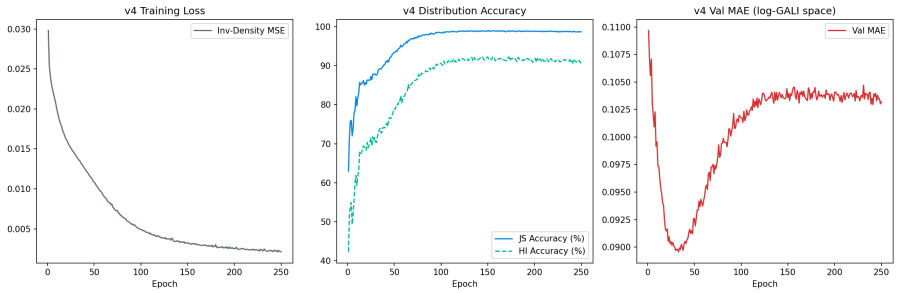

Figure 3: V4 Training Audit. Training and validation loss curves over epochs, along with Jensen-Shannon (JS) divergence tracking.

Manual thresholds were removed and implemented a data-driven, distribution-aware loss function. The GALI dataset was mapped into 200 bins, and each sample was assigned a loss multiplier equal to $1/\sqrt{\text{count}}$. This inverse-density weighting naturally down-weighted the dense chaotic regions and exponentially prioritized the rare, highly-folded transition boundaries without breaking manifold smoothness. Convergence demonstrates stable optimization under inverse-density weighting, with JS divergence approaching over a 99 percent distributional alignment.

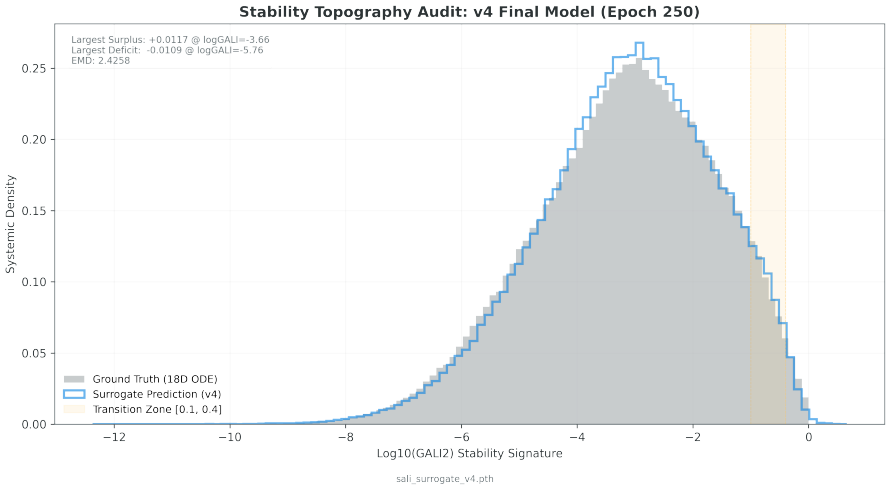

Figure 4: V4 Surrogate Prediction Audit (Inverse-Density Weighted). Distributional alignment between predicted and ground-truth $\log_{10}(\text{GALI})$ values.

Inverse-density weighting significantly improved coverage of low-measure transition regions while still distilling $\log_{10}(\text{GALI})$ values in chaotic regimes. The near-overlap demonstrates successful mitigation of mode collapse and improved manifold continuity. The weight profile used in the final frozen loss function is given as below.

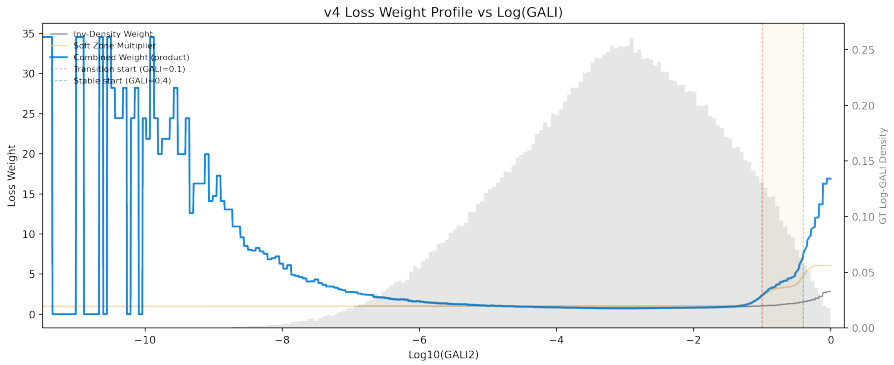

Figure 5: Inverse-Density Weighting Profile (V4). Sample weights assigned as a function of distribution bins of $\log_{10}(\text{GALI})$.

Rare transition regions receive significantly higher weighting, while dense regions are relatively suppressed. This continuous weighting avoids artificial discontinuities introduced by manual thresholding (v3).

Furthermore, v4 tracked Jensen-Shannon (JS) Divergence, achieving a 99.03% JS accuracy. The ONNX-exported v4 model computes in just 0.086 ms.

5 Pillar 6: Residual RL Architecture and Logic

With a real-time stability oracle established, Pillar 6 focused on constructing the Soft Actor-Critic (SAC) residual controller. The physical control vector is $u(t) = u_{PID}(t) + u_{RL}(t)$.

5.1 Multi-Timescale Decoupling

Early configurations placed both the PID and RL at 500 Hz. Because the chaotic system requires physical time to manifest the results of a perturbation, updating the RL agent at 500 Hz caused high-frequency "buzzing" where sequential actions overwrote one another, wasting massive energy.

Solution: We decoupled the operating frequencies. The PID handles high-frequency micro-stabilization at 500 Hz ($dt = 0.005$), while the SAC agent handles "macro" manifold surfing at 50 Hz ($dt = 0.05$).

5.2 Network Representation & Authority

The RL agent only commands 10% to 20% of U_{max}. Because the 9D state variables (including the massive chaotic coordinates) completely dwarfed the tiny residual actions, the SAC Critic was functionally blind to the agent's actions. Injecting `nn.LayerNorm` into the first layers of both the Actor and Critic standardized feature representations, restoring gradient sensitivity to micro-actions.

6 Pillar 6: Hyperparameter Evolution

The journey to simultaneous effort and error reduction required solving complex reinforcement learning economics.

6.1 Reward Economics and Gradient Masking

Initially, the convergence bonus heavily outweighed the effort penalty. To fix this, we transitioned the effort penalty from an L_2 squared norm to an L_1 absolute norm. The squared norm flattened out near zero, removing all gradient pressure to reduce effort. The L_1 absolute uniform norm provided steep, linear gradient pressure pushing back against physical effort all the way to zero.

6.2 Geometric Frustration & Trace Overlap Loss (caSSM Encoder)

To make the agent explicitly aware of the geometry, a Control-Augmented Spectral Submanifold (caSSM) encoder was built and trained. The goal was to align the encoder's latent space with the physical tangent vectors $(\mathbf{w}_1, \mathbf{w}_2)$.

Caveat Encountered: Point-wise alignment gradients conflicted violently with reconstruction gradients, and standard Frobenius norms broke when evaluating higher-dimensional latent spaces.

Solution: A Trace Overlap Loss: $L_{tangent} = 2.0 - \text{tr}(P_{target} P_{encoder})$ was implemented. This measures how much of the target physical subspace is contained within the encoder's subspace, perfectly bounding the loss regardless of dimensionality and allowing the SOAP (Second-Order Adam Preconditioner) optimizer to resolve the gradient conflict.

6.3 Marginal vs. Absolute Effort Traps

We experimented with Marginal Effort rewards (rewarding the agent only when $u_{total} < u_{PID}$). However, this resulted in a stable equilibrium where the agent actively avoided convergence. If it converged, u_{PID} dropped to zero, removing its source of positive reward.

Solution: Absolute effort penalty is mathematically necessary. The agent must first converge (minimizing PID) before $|u_{total}|$ can drop. The final, simplified reward function successfully isolated these objectives into three transparent terms:

1. Distance penalty (linear scaling)
2. Absolute effort penalty (L_1 norm)
3. Strict proximity convergence bonus

7 Conclusion & Future Directions

This project validated the integration of Generalized Alignment Indices into a high-speed neural surrogate to map non-Markovian topological spaces, which subsequently guided a Residual SAC agent to optimize transient dynamics. The final runs demonstrated up to a 46% reduction in L1 control effort and a 74% reduction in tracking error compared to optimally-tuned PID baselines.

8 Technical Stack & Libraries

The framework was built purely in Python using PyTorch for deep learning and continuous control. SciPy was leveraged for Latin Hypercube generation and high-fidelity RK45

numerical integration. ONNXRuntime was utilized for low-level CPU inference speedups, and standard scientific libraries (NumPy, Pandas, Matplotlib) formed the backbone of the analysis pipelines.